

Blatt 5

Kryptologie

Luis Staudt

Performance-Vergleich: DLP – ECDLP

Aufgabenstellung

Verglichen werden soll der Aufwand der **Schlüsselpaar-Erzeugung** (a, A) für zwei auf dem diskreten Logarithmus beruhende Verfahren bei *gleichem Sicherheitsniveau* k gemäß BSI TR-02102-1:

- **Klassisches DLP** in $\text{GF}(p)$: modulare Exponentiation $A := G^a \bmod p$, wobei die Primzahl p eine Bitlänge von N besitzt.
- **ECDLP**: skalare Multiplikation $A := [a]G$ auf einer elliptischen Kurve über $\text{GF}(p)$ mit Körper-Bitlänge n .

Für jedes Sicherheitsniveau k wird ein zufälliger Parametersatz mit Generator G erzeugt, für viele zufällige a der Punkt A berechnet und die Laufzeit gemessen. Die Ergebnisse werden als Boxplots und Vergleichsdiagramme dargestellt und beurteilt.

Sicherheit k	16	32	56	60	64	70	100	120	128	192	256
ECDLP n (Bit)	32	64	112	120	128	140	200	240	256	384	512
GF (p) N (Bit)	64	256	512	700	816	1000	1900	2800	3200	7900	15500

Tabelle 1: Äquivalente Schlüssellängen gemäß BSI TR-02102-1.

Vorgehensweise

1. **Parametersatz (a)**: Pro Bitlänge wird einmalig ein zufälliger Parametersatz erzeugt.
 - $\text{GF}(p)$: zufällige Primzahl p (N Bit) und ein zufälliger Generator $G \in [2, p-1]$.
 - EC: zufällige Primzahl p (n Bit) mit $p \equiv 3 \bmod 4$, zufälliger Kurvenparameter a und ein zufälliger Punkt $G = (x, y)$; der Parameter b wird so gesetzt, dass G auf der Kurve $y^2 = x^3 + ax + b$ liegt.
2. **Berechnung (b)**: Für je einen frischen Zufallswert a (Bitlänge N bzw. n) wird $A := G^a \bmod p$ bzw. $A := [a]G$ berechnet.

3. **Messung (c):** Nach einer JIT-Aufwärmphase werden die Laufzeiten oft wiederholt gemessen (1000 Wiederholungen, für die sehr großen Körper $N \geq 1900$ aus Zeitgründen reduziert auf 200 bzw. 50). Sehr kurze Operationen werden in einer inneren Schleife gebündelt und gemittelt, damit die Messung über der Zeitgeber-Auflösung liegt.

Die Implementierung erfolgt in **Java**. Die modulare Exponentiation nutzt das Standard-Primitiv `java.math.BigInteger.modPow` (Montgomery-Reduktion). Die EC-Punktarithmetik wird **nicht selbst implementiert**, sondern der erprobten und optimierten Bibliothek **Bouncy Castle** (`org.bouncycastle.math.ec`) überlassen; selbstgeschriebene Gruppengesetze sind in einem Benchmark fehleranfällig (Sonderfälle, Formeln) und würden das Ergebnis verfälschen. Lediglich der *Parametersatz* (Primkörper, Kurvenkoeffizient, Generatorpunkt) wird selbst zufällig erzeugt, da die BSI-Bitlängen überwiegend keinen standardisierten Kurven entsprechen. Die statistische Auswertung und die Diagramme werden mit **Python** (`matplotlib`) erzeugt.

```

1 // b so waehlen, dass G = (gx, gy) auf y^2 = x^3 + a*x + b liegt:
2 BigInteger b = gy.multiply(gy)
3   .subtract(gx.modPow(THREE, p)).subtract(a.multiply(gx)).mod(p);
4 ECCurve curve = new ECCurve.Fp(p, a, b, null, null); // Ordnung/Kofaktor irrelevant
5 ECPoint G = curve.createPoint(gx, gy);
6
7 ECPoint A = G.multiply(scalar).normalize(); // A := [a]G

```

Code Listing 1: Parametersatz und skalare Multiplikation via Bouncy Castle

Ergebnisse

Tabelle 2 fasst die Median-Laufzeiten pro Operation zusammen. Der Faktor gibt an, um wie viel die EC-Variante schneller (Faktor > 1) bzw. langsamer (Faktor < 1) als die modulare Exponentiation bei gleichem Sicherheitsniveau ist.

k	n (Bit)	N (Bit)	EC $[a]G$ (μs)	GF(p) G^a (μs)	Faktor
16	32	64	12,8	1,0	0,1×
32	64	256	49,6	8,5	0,2×
56	112	512	171,4	39,1	0,2×
60	120	700	169,8	86,6	0,5×
64	128	816	172,7	133,5	0,8×
70	140	1000	218,8	233,7	1,1×
100	200	1900	464,2	1 377,7	3,0×
120	240	2800	624,2	4 027,5	6,5×
128	256	3200	624,3	5 879,3	9,4×
192	384	7900	1 475,4	88 034,1	59,7×
256	512	15500	2 983,1	658 821,3	220,9×

Tabelle 2: Median-Laufzeiten pro Schlüsselpaar-Erzeugung und Geschwindigkeitsfaktor.

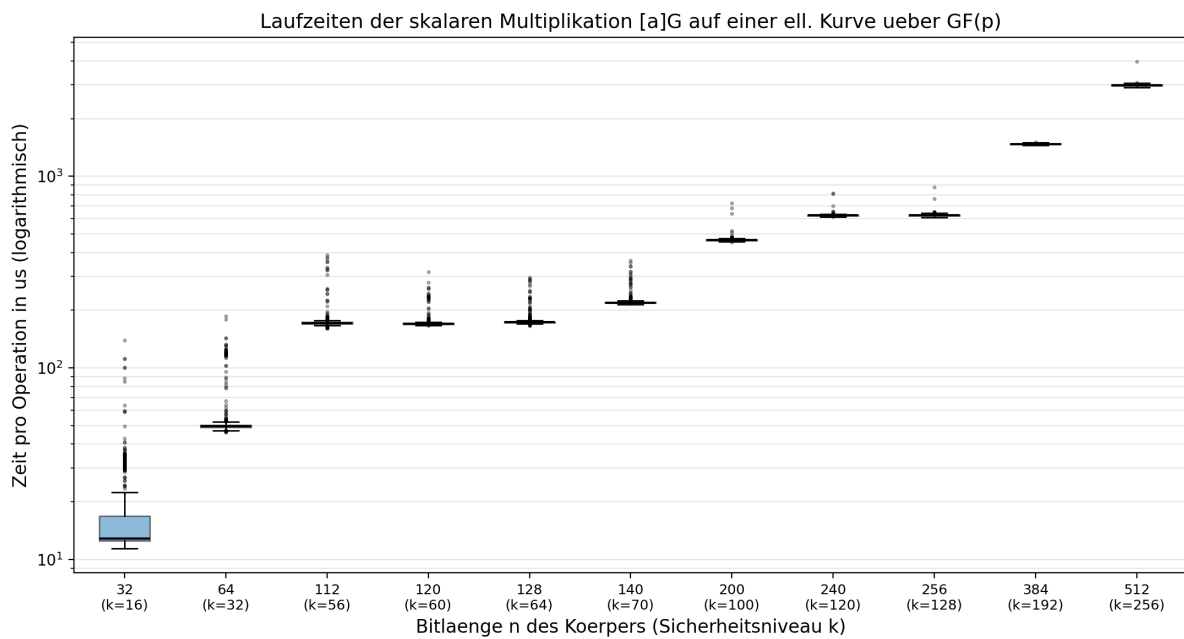


Abbildung 1: Laufzeiten der skalaren Multiplikation $[a]G$ (ECDLP) je Bitlänge n .

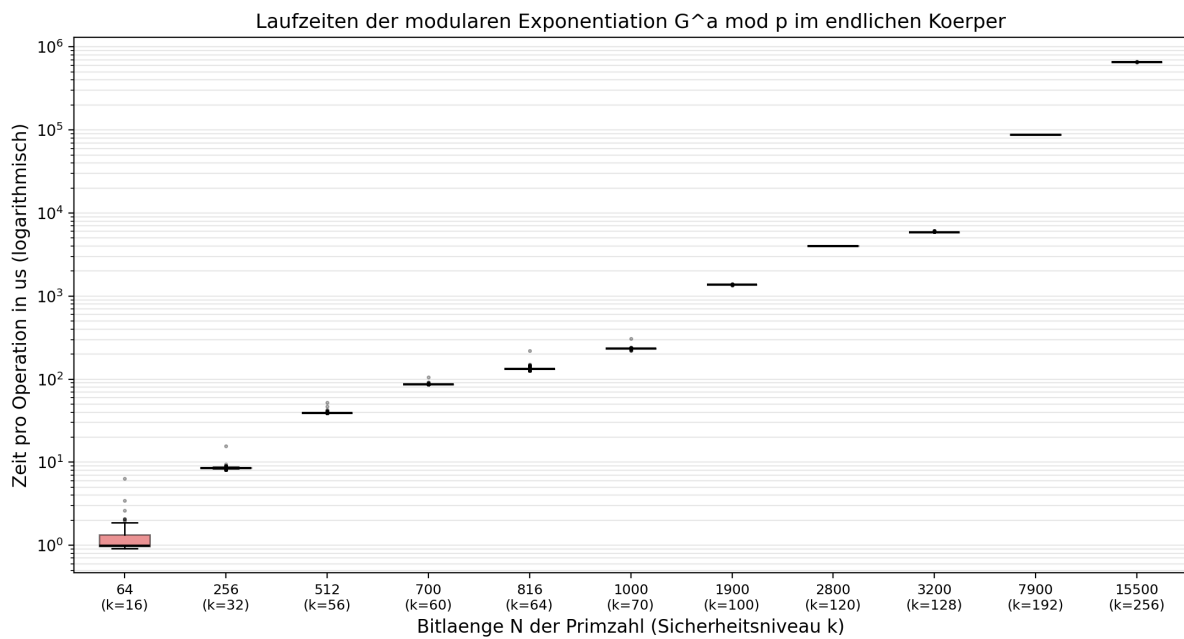


Abbildung 2: Laufzeiten der modularen Exponentiation $G^a \bmod p$ (DLP) je Bitl nge N .

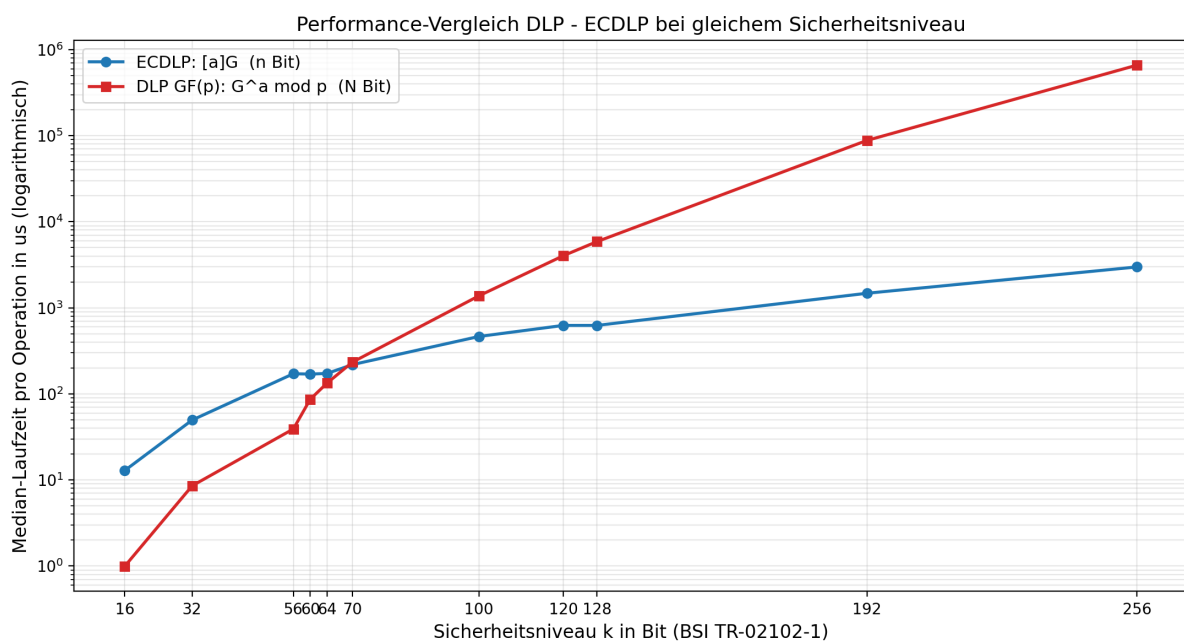


Abbildung 3: Direkter Vergleich der Median-Laufzeiten  ber das Sicherheitsniveau k (log. Skala). Der Schnittpunkt der beiden Kurven liegt bei $k \approx 70$.

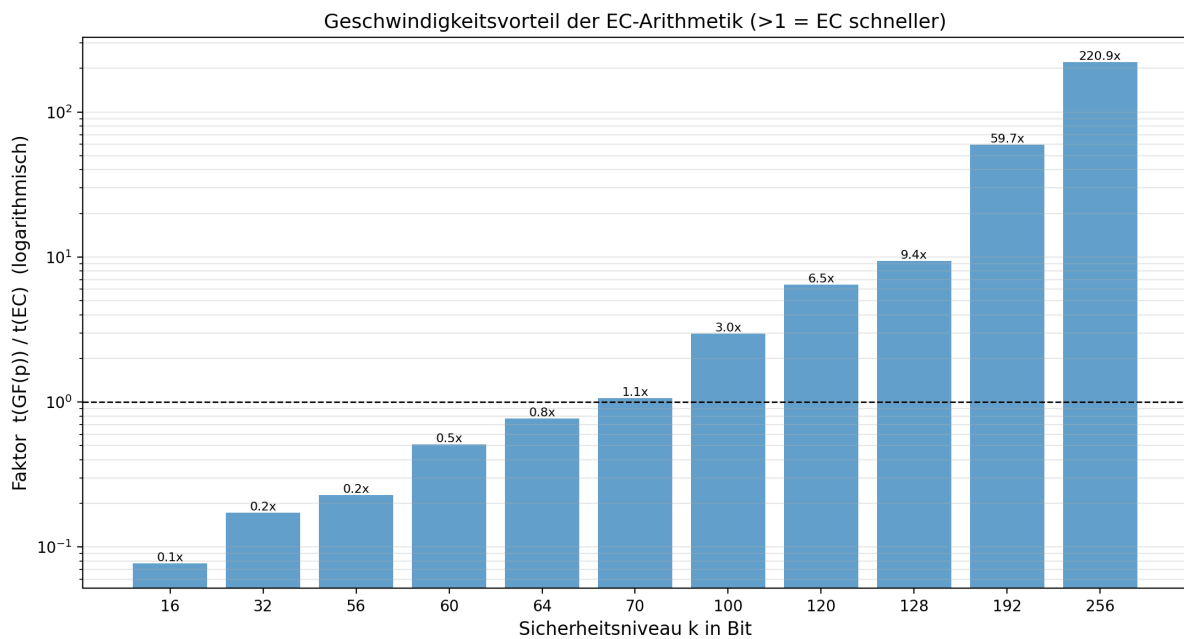


Abbildung 4: Geschwindigkeitsvorteil der EC-Arithmetik: Verhältnis $t(\text{GF}(p))/t(\text{EC})$.

Auswertung

Komplexität

Beide Operationen folgen einem **square-/double-and-add**-Schema mit kubischem Aufwand in der Bitlänge:

- **Modulare Exponentiation** $G^a \bmod p$: ca. N modulare Multiplikationen mit je $\mathcal{O}(N^2) \Rightarrow \mathcal{O}(N^3)$.
- **Skalare Multiplikation** $[a]G$: ca. n Punktoperationen mit je konstant vielen Körpermultiplikationen $\mathcal{O}(n^2) \Rightarrow \mathcal{O}(n^3)$.

Da bei gleichem Sicherheitsniveau $N \gg n$ gilt (z. B. $N = 3200$ gegenüber $n = 256$ bei $k = 128$), ist der Aufwand der EC-Variante asymptotisch um den Faktor $(N/n)^3$ geringer.

Beobachtetes Verhalten

- **Kleine Sicherheitsniveaus** ($k \leq 64$): Hier ist die modulare Exponentiation sogar *schneller*. Die Körper sind klein genug, dass die hochoptimierte `BigInteger.modPow`

(Montgomery-Reduktion) gegenüber dem höheren Konstantenaufwand einer Punktoperation (mehrere Körpermultiplikationen pro Schritt) im Vorteil ist. Bei $k = 16$ ist $\text{GF}(p)$ rund $13\times$ schneller.

- **Schnittpunkt:** Bei $k \approx 70$ ($n = 140$, $N = 1000$) sind beide Verfahren etwa gleich schnell (Faktor 1,1).
- **Praxisrelevante Niveaus ($k \geq 100$):** Ab hier dominiert die EC-Arithmetik klar. Der Vorsprung wächst von $3,0\times$ ($k = 100$) über $9,4\times$ ($k = 128$) bis auf $221\times$ ($k = 256$). Genau dieses überproportionale Auseinanderlaufen wird durch das kubische Wachstum bei stark unterschiedlichen Bitlängen erklärt.

Streuung

Die Boxplots zeigen für beide Verfahren eine sehr geringe Streuung um den Median; die wenigen Ausreißer nach oben sind auf Garbage-Collection und Scheduling der JVM zurückzuführen. Die Laufzeit hängt praktisch nur von der Bitlänge ab und ist gegenüber dem konkreten Zufallswert a nahezu konstant.

Bewertung

Die Messungen bestätigen die in der Einführung genannten Vorteile der EC-Kryptographie quantitativ: Bei *gleichem Sicherheitsniveau* benötigt die elliptische Kurve eine drastisch kürzere Schlüssellänge ($n = 512$ statt $N = 15500$ Bit bei $k = 256$) und ist dadurch bei der Schlüsselpaar-Erzeugung um **mehr als zwei Größenordnungen** schneller.

Der Vorteil ist nicht konstant, sondern wächst mit dem Sicherheitsniveau: Während bei niedrigen Niveaus die ausgereifte Implementierung der modularen Exponentiation noch überwiegt, kehrt sich das Bild ab praxisrelevanten Niveaus ($k \geq 100$) deutlich um. Hinzu kommen die kürzeren Schlüssel, die Speicherplatz und Bandbreite sparen – entscheidende Vorteile auf ressourcenbeschränkten Geräten. Damit erklärt sich, warum moderne Verfahren (z. B. ECDH, ECDSA) elliptische Kurven gegenüber klassischen $\text{GF}(p)$ -Verfahren bevorzugen.

Anmerkung zur Methodik: Es wurde der ungünstigste Fall einer Exponenten-Bitlänge $= N$ gemessen. In Schnorr-/DSA-Gruppen mit Untergruppe der Ordnung $\approx 2k$ Bit wäre der Exponent kürzer; der qualitative Befund (überproportionaler EC-Vorteil bei hohen Niveaus) bliebe jedoch bestehen, da die Kosten einer einzelnen modularen Multiplikation weiterhin $\mathcal{O}(N^2)$ betragen.